



Resumen de Fundamentos, Buenas Prácticas y Patrones Comunes Utilizados en el Proyecto

Este documento resume los **fundamentos técnicos**, las **buenas prácticas** y los **patrones de arquitectura** aplicados en el proyecto Vue 2 + Vuetify + Vuex + Spring Boot.

1. Fundamentos Técnicos Utilizados

Vue.js 2

- Componentes basados en **Options API**.
- Uso de **computed properties** para derivar estado de manera reactiva.
- Uso de **methods** para lógica funcional y manejo de eventos.
- **Ciclos de vida** (`created`, `mounted`) para inicializar datos.
- **Directivas** (`v-if`, `v-for`, `v-model`, `v-show`).

Vue Router

- Arquitectura basada en **Single Page Application (SPA)**.
- Manejo de rutas con parámetros (`/products/:id`).
- **Lazy loading** mediante import dinámico.
- **Route Guards** para protección de rutas según la autenticación.
- Integración con **nprogress** para indicar navegación.

Vuex (Arquitectura Flux)

- Separación de lógica en **state**, **getters**, **mutations** y **actions**.
- Uso de **namespaces** para modularizar el store.
- Concepto de **commit** (mutaciones sincrónicas).
- Concepto de **dispatch** (acciones asincrónicas y lógica de negocio).
- Uso de acciones para consumir APIs mediante Axios.

Vuetify 2

- Uso del sistema de diseño basado en Material Design.
- Componentes como `v-card`, `v-btn`, `v-data-table`, `v-pagination`.
- Slots para crear layouts reutilizables.
- Manejo responsivo con grillas (`v-row`, `v-col`).

Spring Boot Backend

- API REST con controladores.
- CRUD sobre entidad `Product`.
- Uso de repositorio en memoria.
- Manejo de paginación básica.
- CORS habilitado para comunicación local.

Axios

- Cliente HTTP para consumo del backend.
 - Uso dentro de acciones Vuex.
-

2. Principios de Buenas Prácticas Aplicados

Separación de responsabilidades

Frontend, backend, estado global y vistas se mantienen desacoplados.

Vuex como única fuente de verdad global

Los componentes no manipulan estado compartido directamente.

Mutaciones son siempre sincrónicas

Para garantizar trazabilidad del estado.

Acciones manejan lógica asincrónica

Incluye llamadas a API, validaciones y post-procesamiento.

Componentes pequeños y reutilizables

Layouts separados, vistas especializadas.

Uso de lazy loading

Optimiza el performance de la SPA cargando vistas solo cuando son necesarias.

Código declarativo en templates

Evita lógica compleja en el template, se traslada a computed/methods.

Manejo explícito de errores de navegación

Uso de `.catch(() => {})` o parche global de router.

Nombramiento consistente y legible

Uso de camelCase para métodos, SNAKE_CASE para mutaciones.

Evitar duplicación

Uso de mapActions, mapGetters, computed setters.

Rutas protegidas por autenticación

El guard analiza `meta.requiresAuth` para controlar el flujo.

3. Patrones Comunes de Desarrollo Aplicados

3.1 Patrones del ecosistema Vue

Pattern: Smart vs Dumb Components

- Vistas (smart) consumen Vuex.
- Componentes UI (dumb) reciben props y emiten eventos.

Pattern: Computed Setter

Usado en paginación para convertir índices 0-based ↔ 1-based.

Pattern: Vuex Flux Architecture

- State: fuente de verdad.
- Getters: derivaciones.
- Mutations: cambios sincrónicos.
- Actions: cambios asincrónicos.

Pattern: Centralized Router Guard

Valida autenticación antes de permitir el acceso.

Pattern: API service encapsulated in Vuex

Los componentes nunca llaman axios directamente.

3.2 Patrones de arquitectura backend

Pattern: Controller - Repository

Controladores manejan solicitudes, repositorio maneja datos.

Pattern: DTO para paginación

Entrega limpia de datos al frontend (`ProductPageDTO`).

3.3 Patrones de diseño generales

DRY (Don't Repeat Yourself)

- Acciones reutilizan lógica.
- Computed evitan duplicar procesamiento.

Single Source of Truth

Estado global reside en Vuex y no en componentes.

Separation of Concerns

- Router maneja navegación.
- Vuex maneja estado.
- Componentes manejan UI.

KISS (Keep It Simple, Stupid)

Componentes con lógica mínima y clara.

Fail Fast

Guardias y validaciones tempranas.

4. 🛠️ Conclusión

Este proyecto integra los principios fundamentales de un desarrollo frontend moderno usando Vue 2, combinados con buenas prácticas del ecosistema SPA, manejo de estado centralizado, modularidad, patrones de diseño y un backend limpio en Spring Boot. El resultado es una arquitectura clara, mantenible, escalable y alineada con prácticas profesionales.

Si deseas extender este documento con un apartado más técnico o agregar diagramas de arquitectura, puedo generarlos también.